PROJECT REPORT

# PERSONAL EXPENSE TRACKER & ANALYSIS SYSTEM

*A Python-based application for recording, analysing, and visualising personal expenses*


| | |
|---|---|
| **Student Name** | Archit chatterjee |
| **Roll Number** | 26102410025 |
| **Department** | Computational sciences (MCA) |
| **Subject Name** | Problem Solving Using Python |
| **Subject Code** | MCA17400 (P) |
| **Academic Session** | 2026-2028 |


*Submitted in partial fulfilment of the requirements of the subject stated above.*

## Report Preparation Note

This report has been prepared from the two Python files supplied for review. Both files contain the same Personal Expense Tracker & Analysis System implementation, so they are treated as duplicate copies of one project rather than as two separate projects.

The report describes the functionality visible in the source code. No separate CSV dataset, execution logs, screenshots, institutional details, student details, or measured performance results were supplied. Accordingly, the testing section distinguishes test cases proposed for verification from results that have actually been observed. Replace the bracketed cover-page fields and add genuine screenshots or execution evidence before final submission.

## Table of Contents


- 1. Introduction
- 2. Problem Statement
- 3. Objectives
- 4. Scope of the Project
- 5. Literature Review / Existing System
- 6. Proposed System
- 7. Project Methodology
- 8. Project Internal Structure
- 9. Implementation
- 10. Testing and Results
- 11. Conclusion
- 12. Future Scope
- 13. References

# 1. Introduction

Managing day-to-day expenses is an important part of personal financial planning. When spending information is recorded informally or not recorded at all, it can be difficult to understand how much money is being spent, which categories account for the largest share, and how spending changes from month to month.

The Personal Expense Tracker & Analysis System is a menu-driven, command-line application developed in Python. It allows a user to enter an expense with a date, category, amount, and optional description. The application stores records in a CSV file so that the information remains available between program runs. It uses Pandas to load, organise, sort, group, and analyse expense records, and Matplotlib to generate visual representations of spending.

The system supports adding and viewing expenses, sorting records, generating an overall spending summary, producing a report for a selected month, and creating four types of charts. The application is intended as a small, local personal-finance utility rather than a banking or accounting platform.

# 2. Problem Statement

Individuals may find it difficult to maintain a clear and consistent record of routine expenses. Manual notes or scattered records can make it time-consuming to calculate total expenditure, compare categories, identify the highest and lowest transactions, or examine monthly spending trends.

The project addresses the following problem: how can an individual record personal expenses in a simple local application and obtain useful summaries and visual analysis without requiring a remote service or a complex financial-management system?

The supplied implementation addresses this by providing structured expense entry, CSV-based persistence, summary calculations, month-specific reporting, sorting, and chart generation.

# 3. Objectives

- Provide a simple menu-based interface for recording personal expenses.
- Capture the date, category, amount, and an optional description for each expense.
- Validate user input, including accepted date formats and positive finite monetary amounts.
- Store expense records in a CSV file and load them when the application starts.
- Allow users to view all recorded expenses and sort them by date, amount, or category.
- Calculate total and average expenditure, monthly totals, category-wise totals and percentages, and the highest and lowest expenses.
- Generate a report for a selected month.
- Visualise expenditure using bar, pie, line, and histogram charts.

# 4. Scope of the Project

## 4.1 Functional coverage

- Expense entry with optional description.
- Date entry using YYYY-MM-DD, DD-MM-YYYY, or DD/MM/YYYY; pressing Enter selects the current date.
- Category selection from Food, Travel, Education, Shopping, Entertainment, and Other.

- CSV file storage in the same directory as the Python script.
- Viewing and sorting expense records.
- Overall and month-specific spending summaries.
- Chart generation and saving chart images in the project directory.

### 4.2 Target users

The intended users are individuals who want to maintain a basic personal expense record on a computer. The interface assumes that the user can run a Python program and interact with a terminal.

### 4.3 Limitations

- The application is command-line based; it does not provide a graphical or web interface.
- Data is stored locally in a CSV file. The supplied code does not implement user accounts, authentication, encryption, cloud synchronisation, or multi-user access.
- The categories are predefined in the source code.
- The application does not record income, budgets, account balances, recurring payments, or bank transactions.
- The supplied code does not include edit or delete functionality for existing records.
- Charts and reports depend on expense data already being recorded.
- No sample CSV file or measured execution/performance results were supplied with the project files.

## 5. Literature Review / Existing System

The supplied project does not include a formal literature review or a survey of named commercial products. This section therefore describes the general approaches relevant to the problem and compares them with the features implemented in the source code.

### 5.1 Manual record keeping

A person can write expenses in a notebook or maintain a handwritten list. This approach is straightforward, but totals, category comparisons, and monthly trends must generally be calculated manually. Records may also be difficult to sort or analyse consistently.

### 5.2 Spreadsheet-based tracking

A spreadsheet can store rows and columns of expense information and can support sorting, formulas, and charts. However, the user must maintain the spreadsheet structure and formulas. The supplied Python application automates common entry, summary, and chart-generation steps through a fixed menu.

### 5.3 General expense-tracking applications

Dedicated expense applications may offer graphical interfaces, accounts, synchronisation, budgeting, and integrations. Those capabilities are outside the scope of the supplied code. The project instead focuses on a small local program with a transparent CSV data file and a limited set of analysis functions.

### 5.4 Identified gap

The project provides a compact workflow for entering expense records and analysing them with Python libraries. Its focus is simplicity and local storage, not the broader financial services or account-management capabilities that may be available in larger applications.

# 6. Proposed System

The proposed system is a Python command-line application that combines structured expense entry, local CSV storage, tabular processing, summary calculations, and chart visualisation.

## 6.1 Key features

| Feature | Description |
|---|---|
| Add expense | Collects date, category, amount, and optional description, then saves the new record. |
| View expenses | Displays all records in a readable table. |
| Sort expenses | Sorts records by date, amount, or category in ascending or descending order. |
| Spending summary | Displays record count, total, average, monthly expenditure, category totals and percentages, and highest/lowest expense. |
| Monthly report | Filters records for a selected YYYY-MM month and displays total, average, and category totals. |
| Charts | Creates a category bar chart, category-share pie chart, monthly trend line plot, or amount histogram. |

## 6.2 Expected outcome

When used with valid expense records, the application is expected to provide a persistent record of expenses and help the user inspect spending patterns through numerical summaries and charts.

# 7. Project Methodology

## 7.1 Development approach

The code is organised in functional sections: configuration, file handling, input validation, core expense operations, visualisation, and the main menu. Each major task is implemented as a separate function. The main menu calls the appropriate function according to the user's selection.

## 7.2 Technologies and tools

| Technology / Tool | Role in the project |
|---|---|
| Python | Main programming language; implements application logic, input handling, file operations, and program flow. |
| Pandas | Reads and writes CSV data and performs tabular operations such as sorting, grouping, aggregation, and filtering. |
| Matplotlib | Creates the bar chart, pie chart, monthly line plot, and histogram. |
| CSV | Plain-text, tabular file format used for persistent expense storage. |
| datetime | Parses dates, supplies the current date, and supports month-based reporting. |
| os | Builds file paths and checks whether the CSV file exists. |
| math | Checks that entered amounts are finite numbers. |
| Terminal / command line | Interface through which the user selects menu options and enters data. |

### 7.3 Dependencies and execution

The source file identifies Pandas and Matplotlib as external dependencies. They can be installed using the command below. The program is started by running the Python script.

```
pip install pandas matplotlib
python expense_tracker.py
```

### 7.4 Data format

The CSV file is named expenses.csv and is created or accessed in the same directory as the Python script. The four columns defined by the program are Date, Category, Amount, and Description. Dates are written in YYYY-MM-DD format when saving.

## 8. Project Internal Structure

### 8.1 High-level architecture

| Layer / Component | Responsibility | Main functions / elements |
|---|---|---|
| User interface | Displays menu prompts and receives user choices and expense details. | main(), MENU, ask_date(), ask_category(), ask_amount() |
| Application logic | Coordinates expense entry, viewing, sorting, summaries, and monthly reports. | add_expense(), view_expenses(), sort_expenses(), summary(), monthly_report() |
| Data processing | Loads, validates, transforms, groups, and saves tabular records. | load_expenses(), save_expenses(), Pandas DataFrame operations |
| Storage | Persists expense records between executions. | expenses.csv |
| Visualisation | Builds and saves spending charts. | bar_chart(), pie_chart(), line_plot(), histogram(), finish_plot() |

Architecture flow:

```
User
  |
  v
Main menu (main)
  |
  +--> Input and validation helpers
  |
  +--> Expense operations and reports
  |         |
  |         v
  |     Pandas DataFrame
  |         |
  |         v
  |     CSV storage (expenses.csv)
  |
  +--> Chart functions --> Matplotlib --> PNG chart files
```

### 8.2 Module-wise structure

| Module / Section | Functions / constants | Purpose |
|---|---|---|
| Configuration | DATA_FILE, COLUMNS, CATEGORIES, CURRENCY, DATE_FORMATS | Defines file location, schema, categories, currency label, and accepted date |

| | | |
|---|---|---|
| | | formats. |
| File handling | load_expenses(), save_expenses() | Reads records from CSV and writes the current DataFrame back to CSV. |
| Input helpers | parse_date(), ask_date(), ask_amount(), ask_category() | Validates and normalises the information entered by the user. |
| Core features | add_expense(), has_data(), show_table(), view_expenses(), sort_expenses(), summary(), monthly_report() | Implements expense entry, display, sorting, and analysis. |
| Visualisation | finish_plot(), bar_chart(), pie_chart(), line_plot(), histogram(), charts_menu() | Creates and saves charts using Matplotlib. |
| Program entry point | MENU, main(), __main__ block | Displays the menu, routes user selections, and handles keyboard interruption / end-of-input. |

## 8.3 Workflow

```
START
  |
  v
Load expenses.csv (or create an empty DataFrame if absent)
  |
  v
Display main menu
  |
  +-- 1: Add expense --> Validate inputs --> Save CSV --> Menu
  +-- 2: View expenses -------------------------------> Menu
  +-- 3: Sort records --------------------------------> Menu
  +-- 4: Spending summary ----------------------------> Menu
  +-- 5: Monthly report ------------------------------> Menu
  +-- 6: Select chart --> Generate / save chart ------> Menu
  +-- 0: Exit
  |
  v
END
```

## 8.4 Data design

| Field | Meaning | Representation / handling |
|---|---|---|
| Date | Date on which the expense occurred. | Accepted from three input formats; saved as YYYY-MM-DD and parsed as datetime when loaded. |
| Category | Type of expense. | One of Food, Travel, Education, Shopping, Entertainment, or Other. |
| Amount | Expense value. | Converted to a finite positive number and rounded to two decimal places. |
| Description | Optional note about the expense. | Text; missing descriptions loaded from CSV are replaced with an empty string. |

The project uses a flat CSV structure rather than a relational database. No database server, SQL schema, primary key, or foreign-key relationship is present in the supplied code.

# 9. Implementation

## 9.1 Configuration and file handling

The application defines the CSV path relative to the script location, the four data columns, six fixed categories, the display currency label Rs., and three accepted date formats. load_expenses() returns an empty DataFrame when the file does not exist. If the file exists, it is read with Pandas; date and amount columns are converted, missing descriptions are filled, and rows with invalid dates or amounts are dropped. save_expenses() formats dates as YYYY-MM-DD and writes the DataFrame to CSV without a row index.

## 9.2 Input validation

parse_date() attempts each accepted date format and returns no date when parsing fails. ask_date() repeats the prompt until the date is valid, defaults to today's date when the user presses Enter, and rejects future dates. ask_amount() removes commas, converts the input to a floating-point number, rejects non-numeric, non-finite, zero, and negative values, and rounds accepted amounts to two decimal places. ask_category() accepts either a category number or a case-insensitive category name.

## 9.3 Adding and viewing expenses

add_expense() collects the four fields, constructs a one-row DataFrame, appends it to the existing records (or uses it as the first row), saves the updated data, and prints a confirmation. view_expenses() checks whether data exists and then displays a formatted table. Amounts are shown with two decimal places.

## 9.4 Sorting

sort_expenses() lets the user choose Date, Amount, or Category. It then asks for ascending or descending order and uses DataFrame.sort_values() to display the sorted result. The supplied function displays the sorted result; it does not save the changed ordering back to the CSV file.

## 9.5 Spending summary

summary() calculates the number of expense records, total expenditure, and average expenditure per record. It groups amounts by calendar month to calculate monthly totals and their average. It also groups by category, sorts category totals in descending order, and calculates each category's percentage of the category-total sum. Finally, it identifies the rows with the maximum and minimum amount and displays their details.

## 9.6 Monthly report

monthly_report() prompts for a month in YYYY-MM format and defaults to the current month when the user presses Enter. It filters records to the selected month. If there are matching records, it displays the total, average, and category-wise totals; otherwise, it reports that no expenses were found for that month.

## 9.7 Chart generation

| Chart | Implementation and purpose | Output filename |
|---|---|---|
| Bar chart | Groups amounts by category, sorts totals from highest to lowest, and labels bars with values. | chart_bar.png |
| Pie chart | Shows the share of total recorded spending represented by each category. | chart_pie.png |
| Line plot | Groups spending by month and plots the monthly totals as a trend. | chart_line.png |
| Histogram | Shows the distribution of individual expense amounts using bins. | chart_histogram.png |

finish_plot() applies a tight layout, saves the chart at 120 DPI in the project directory, prints the saved path, and displays the chart. charts_menu() checks that expense data exists before offering the four chart options.

## 9.8 Main menu and program control

main() loads the expense records once at startup, displays the number of loaded records, and repeatedly displays the menu. Choices 1–6 call the corresponding feature, while choice 0 exits. Invalid menu selections produce an error message. The program's entry-point block calls main() and handles KeyboardInterrupt and EOFError by printing an exit message.

```
 architchatterjee@ArchitMac  ~  Documents  Python _Project  main  Python _Project 3.15.0
❯ python expense_tracker.py
Loaded 4 expense record(s) from /Users/architchatterjee/Documents/Python _Project/expenses.csv

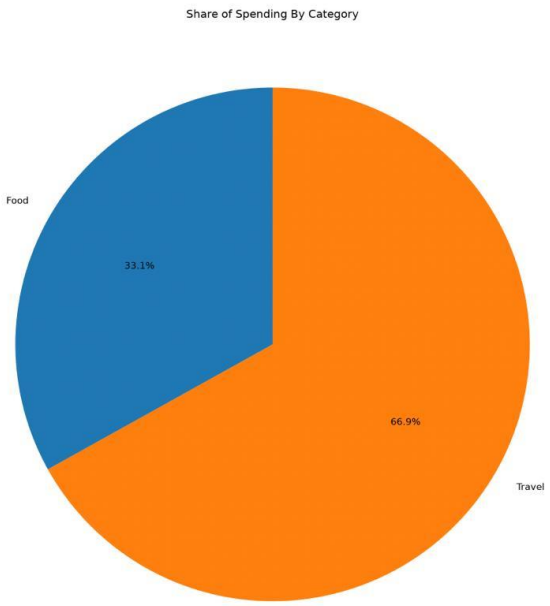
========== EXPENSE TRACKER ==========
 1. Add expense
 2. View all expenses
 3. Sort expenses
 4. Spending summary (total, monthly, category, average, highest/lowest)
 5. Report for a specific month
 6. Charts
 0. Exit
=====================================

Select an option:
```

```
--- All Expenses ---
      Date Category    Amount    Description
2026-10-29     Food 10,000.00     Vegitables
2026-10-30   Travel 30,000.00           rome
2026-10-30     Food  4,500.00 Italian cusine
2026-11-06     Food    333.00        qwswasa

========== EXPENSE TRACKER ==========
 1. Add expense
 2. View all expenses
 3. Sort expenses
 4. Spending summary (total, monthly, category, average, highest/lowest)
 5. Report for a specific month
 6. Charts
 0. Exit
=====================================

Select an option:
```


Share of Spending By Category
Food
33.1%
66.9%
Travel

# 10. Testing and Results

The supplied files contain application code but do not include a test dataset, test log, screenshots, or recorded output from an execution session. The following table is therefore a proposed functional test plan derived from the implemented behaviour. The cases should be executed and the actual outcome recorded before the report is submitted as a tested-results report.

| Test ID | Test scenario / input | Expected behaviour from code | Evidence status |
|---|---|---|---|
| TC-01 | Start when expenses.csv does not exist. | Application starts with an empty DataFrame and reports zero loaded records. | Not executed here |
| TC-02 | Add an expense using a valid date, category, positive amount, and description. | A new record is appended and saved to expenses.csv; a save confirmation is printed. | Not executed here |
| TC-03 | Press Enter at the date prompt. | Today's date is used. | Not executed here |
| TC-04 | Enter an invalid date or a future date. | The program displays a validation message and prompts again. | Not executed here |
| TC-05 | Enter text, zero, a negative number, NaN, or infinity as amount. | The amount is rejected and the program prompts again. | Not executed here |
| TC-06 | Enter a valid category number or name. | The corresponding predefined category is returned. | Not executed here |
| TC-07 | View records when no expenses exist. | A message indicates that no expenses have been recorded. | Not executed here |
| TC-08 | Sort records by date, amount, or category. | The selected sort order is displayed; descending order is used when d is entered. | Not executed here |
| TC-09 | Request the spending summary with records present. | Record count, total, average, monthly totals, category totals/percentages, and highest/lowest records are displayed. | Not executed here |
| TC-10 | Request a month with matching records and a month without matching records. | The matching month report is displayed, or a no-records message is shown. | Not executed here |
| TC-11 | Generate each of the four chart types with data present. | The selected chart is displayed and saved using its corresponding PNG filename. | Not executed here |
| TC-12 | Select an invalid menu option. | An invalid-option message is displayed and the menu loop continues. | Not executed here |

## 10.1 Results and observations

From inspection of the source code, the implemented feature set includes persistent CSV storage, input validation, record display and sorting, aggregate spending analysis, month-specific reporting, and four chart options. These are code-level observations, not claims that every test case has been run successfully.

No numerical totals, chart screenshots, execution time, memory measurements, or performance benchmarks can be reported because no actual expense dataset or execution evidence was supplied. Add screenshots of a real run and record the observed results for the test cases above before submission.

### 10.2 Suggested screenshots to insert

- Screenshot 1: Main menu displayed in the terminal.
- Screenshot 2: Adding an expense, including the successful save confirmation.
- Screenshot 3: View all expenses showing sample records.
- Screenshot 4: Spending summary output.
- Screenshot 5: Monthly report output.
- Screenshot 6: One or more generated charts (bar, pie, line, histogram).

## 11. Conclusion

The Personal Expense Tracker & Analysis System demonstrates how Python can be used to build a simple local application for recording and analysing personal expenses. The program accepts structured expense information, validates key inputs, stores records in a CSV file, and uses Pandas to display, sort, group, and summarise the data. Matplotlib is used to generate four different visualisations.

The implemented functions address the main objectives of basic expense recording, persistence, summary calculation, monthly reporting, and visual analysis. The project is suitable as an introductory application demonstrating Python functions, file handling, data processing, validation, and plotting. Its current command-line interface and local CSV storage keep the design simple, while also defining clear areas for future enhancement.

## 12. Future Scope

- Add functionality to edit and delete existing expense records.
- Introduce user-defined categories and category management.
- Develop a graphical user interface or web-based interface.
- Add budget limits and alerts when spending approaches or exceeds a budget.
- Include income tracking and calculate net savings or cash flow.
- Add date-range filtering and exportable PDF or Excel reports.
- Provide recurring-expense support and reminders.
- Add data backup, restore, and optional cloud synchronisation.
- Introduce authentication and stronger data protection if the application is extended for multiple users.
- Add automated unit tests and integration tests, and measure performance with larger datasets.

These items are recommendations for future development; they are not implemented in the supplied source files.

## 13. References

The following references identify the language and libraries used by the supplied implementation. They are documentation references for the technologies, not evidence of additional features in the project.


1. Python Software Foundation. Python 3 Documentation. https://docs.python.org/3/
2. Pandas Development Team. pandas Documentation. https://pandas.pydata.org/docs/
3. Matplotlib Development Team. Matplotlib Documentation. https://matplotlib.org/stable/
4. Python Software Foundation. datetime — Basic date and time types. https://docs.python.org/3/library/datetime.html
5. Python Software Foundation. csv — CSV File Reading and Writing. https://docs.python.org/3/library/csv.html

Primary project source: expense_tracker.py (the two uploaded copies were identical in the reviewed content).